

Elective Surgery Scheduling

An interval-based constraint program for weekly OR planning

Hector Bonilla

Agenda

- 1 Problem & Motivation
- 2 Scope & Assumptions
- 3 The Model
- 4 Why CP-SAT, Not a Bigger MIP
- 5 Architecture & Reusability
- 6 Open Questions & Next Steps

Backup slides (full math for both models) are linked directly from the relevant slide – click through any time, not just at the end.

The problem

One hospital, one week, an elective waiting list bigger than the week's capacity.

- Decide, for each waiting case: which room, what time, which day, which surgeon – or leave it for a later week.
- Has to be **feasible**: rooms, surgeons, and a handful of shared resources are all finite.
- Has to be **fair**: a genuinely urgent patient shouldn't sit behind a routine one just because the routine case reached the front of a FIFO queue first.

The problem

One hospital, one week, an elective waiting list bigger than the week's capacity.

- Decide, for each waiting case: which room, what time, which day, which surgeon – or leave it for a later week.
- Has to be **feasible**: rooms, surgeons, and a handful of shared resources are all finite.
- Has to be **fair**: a genuinely urgent patient shouldn't sit behind a routine one just because the routine case reached the front of a FIFO queue first.

Brief's own framing: room hours, surgeon availability, shared equipment, turnover between cases, downstream bed pressure – explicitly invites simplification, not a full hospital simulation.

Why it matters – this isn't a toy problem

- **OR time is expensive and perishable.** ~ \$30–40 per minute once staffing/equipment/overhead are amortized, one widely cited U.S. estimate (Macario, 2010) – and an idle OR-minute today can't be banked for tomorrow.
- **The waiting list is the audited number.** 16% of ~7,400 patients in one hospital's 2016 audit had already breached their clinical deadline, by 147 days on average (Marques & Captivo, 2015). Throughput alone can look good while failing exactly the patients the system exists to protect.
- **This is a recognized sub-problem.** Cardoen, Demeulemeester & Beliën (2010) frame “advance scheduling” – deciding which case runs on which day – as distinct from same-day sequencing or disruption handling. That's the scope of this model.
- **Elective and emergency demand compete for the same rooms** (Van Riet & Demeulemeester, 2015) – the reason an emergency-add-on priority tier exists in the model at all.

What's kept, what's left out

Kept: case→room→time assignment, surgeon availability & workload limits, room/service rosters, one shared piece of equipment, a downstream recovery/ICU-bed constraint.

Left out, deliberately:

- Stochastic durations
- Same-day disruption
- Nurses/anaesthetists as separate resources
- Sequence-dependent cleaning
- Multi-week planning
- An explicit emergency capacity reserve

Key assumptions (1/2)

- 1 **Deterministic durations.** One estimated duration per case. Real durations are stochastic and biased *long* – the standard tractable starting point (Denton et al., 2010); stochastic extension named, not attempted.
- 2 **Turnover depends on the case's own duration**, bucketed (15/25/40 min), not on which two cases are adjacent. The sequence-dependent piece is real but secondary here (CP Optimizer appendix shows how it would work).
- 3 **Surgeons are the binding staffing resource.** Nurses/ anaesthetists assumed bundled with the room's roster – true whenever the surgeon's calendar, not support staff's, is the actual bottleneck.
- 4 **One occurrence per patient per week.** Conservative default; combining same-day multi-procedure cases is service-specific clinical judgment, not assumed here.

Key assumptions (2/2)

- 5 **Bed capacity is constant across the week**, with an *explicit* overflow penalty for a stay crossing the horizon boundary – not a silent approximation either way.
- 6 **One ad hoc institutional rule (pediatric block)** included as a worked example: costs nothing structurally, just one more eligibility predicate.
- 7 **Single week, single hospital, no same-day disruption, no explicit emergency reserve.** A genuinely different problem (reactive rescheduling) from advance scheduling.

Full reasoning, trade-offs, and what relaxing each one would require: FORMULATION.md §2.

Sets and indices

Symbol	Meaning
$c \in C$	Surgical cases (one per patient-procedure pair)
$d \in D$	Planning days, $D = \{1, \dots, 5\}$
$r \in R$	Operating rooms
$h \in H$	Surgeons
$e \in E$	Shared equipment types
$D_c \subseteq D$	Days c may run on ($\{1\}$ if must-run-today)

Key parameters

Symbol	Meaning
$t_c^{op}, t_c^{clean}, t_c^{tot}$	Operative / turnover / total room time
k_{dr}, k_{hd}, k_h	Room day capacity; surgeon daily / weekly limit
$p_c, wl_c, wl_p^{max}, dd_c$	Priority; days waited; max wait; slack to deadline
μ_p, α, w_c	Priority multiplier; urgency multiplier; non-scheduling penalty
u_{ce}, κ_{ed}	Equipment need; equipment capacity
$\rho(c), los_c, \beta_\rho, \pi^{ovf}$	Bed pool; length of stay; bed capacity; overflow penalty

► Full table + where every default comes from (backup)

Decision variables (CP-SAT)

For every (c, d, r) surviving eligibility (room-service roster, ambulatory-only, pediatric block, surgeon availability):

$$\text{pr}_{cdr} \in \{0, 1\} \quad \text{start}_{cdr} \in [0, k_{dr}]$$

Two interval variables, same start, different sizes:

$$\text{iv}_{cdr} : [\text{start}_{cdr}, \text{start}_{cdr} + t_c^{\text{tot}}) \quad (\text{ROOM}) \quad \text{sgiv}_{cdr} : [\text{start}_{cdr}, \text{start}_{cdr} + t_c^{\text{op}}) \quad (\text{SURGEON})$$

$$u_c \in \{0, 1\}, \quad \sum_{d,r} \text{pr}_{cdr} + u_c = 1$$

A room stays blocked through cleaning; the surgeon is free the moment the operation ends and can scrub into a different room. One shared start per candidate is what lets the two NoOverlap checks (room, surgeon) propagate against each other without an explicit link. [▶ Full variable detail \(backup\)](#)

Objective function

$$\begin{aligned}
 \min \quad & \underbrace{\sum_{c: dd_c \geq 0} \sum_{d \in D_{c,r}} [dd_c + d] pr_{cdr}}_{\text{on-time, prefer earlier}} + \underbrace{\sum_{c: dd_c < 0} \sum_{d \in D_{c,r}} [dd_c + \alpha d] pr_{cdr}}_{\text{overdue, urgency-weighted}} \\
 & + \underbrace{\sum_{c: p_c \neq 4} w_c u_c}_{\text{non-scheduling penalty}} + \underbrace{\pi^{\text{ovf}} \sum_{c: \rho(c) \neq \text{none}} \text{overflow}_c}_{\text{bed-overflow penalty}} \\
 & w_c = \mu_{p_c} \cdot \text{PenaltyCurve}(dd_c) + 1.2 \cdot \max_{c'} dd_{c'}
 \end{aligned}$$

Term 3 dominates Terms 1–2 by construction (the displacement is derived, not guessed) – the model only drops a case when there's genuinely no room for it. [▶ \$w_c\$ in full, and the margin proof \(backup\)](#)

Constraints, at a glance

- **C1–C3** – one occurrence/patient/week; priority-4 locked to day 1; everyone else scheduled-or-penalized.
- **C4–C6** – eligibility (room-service roster, ambulatory-only, pediatric block) – pre-filters, not constraint rows.
- **C7** – room: exact non-overlap, not a capacity sum.
- **C8** – surgeon: exact non-overlap on the surgeon's own window, plus a daily-minutes cap.
- **C9** – surgeon weekly time limit.
- **C10** – shared equipment: exact time-overlap check, not a daily headcount.
- **C11** – recovery/ICU beds: a multi-day interval starting on the day of surgery.

▶ C1–C11 in full (backup)

The core argument

“Total minutes used today $\leq$ capacity” is not the same statement as “these cases don’t collide”

For one room, a set of non-overlapping durations can always be packed sequentially – the two statements coincide. They stop coinciding the moment a resource is *shared across rooms* (this project’s C-arm).

The core argument

“Total minutes used today $\leq$ capacity” is not the same statement as “these cases don’t collide”

For one room, a set of non-overlapping durations can always be packed sequentially – the two statements coincide. They stop coinciding the moment a resource is *shared across rooms* (this project’s C-arm).

- NoOverlap / Cumulative are global constraints with their own polynomial-time propagation (Vilím, 2004; Schutt et al., 2009) – they prune from the problem’s time structure directly.
- The MIP alternative (pairwise big-M disjunctions) is quadratic in conflicting pairs, and the big-M constants weaken the LP relaxation as that count grows (Baptiste, Le Pape & Nuijten, 2001).
- CP-SAT’s parallel portfolio search is the actual state of the art here – no hand-rolled branching strategy on top of it.

Proof, not assertion – the demo instance

One shared C-arm, capacity 1, needed by 4 cases.

Solver	Status	Objective	Scheduled
CP-SAT (primary)	Optimal	155.0	20/20
OR-Tools/CBC (comparison MIP)	Optimal	157.0	20/20

CP-SAT: Mon=[C19]
 Tue=[C15,C17]
 Wed=[C14]

CBC: Mon=[C19] Tue=[C15]
 Wed=[C17] Thu=[C14]

CP-SAT checks literal time overlap and legally puts two C-arm cases on the *same* day;
 CBC's day-count cap forbids that outright – a smaller feasible region, not a worse search.

Proof at scale – 200 cases, 12 rooms, 17 surgeons

Solver	Status	Objective	Own gap	vs. true MIP opt.	Sched.
Gurobi (true MIP optimum)	Optimal	74,074.0	0.00%	—	130/200
OR-Tools/CBC (2 min)	Feasible	74,116.0	0.56%	+0.06%	130/200
CP-SAT (2 min)	Feasible	69,956.0	6.62%	−5.56%	131/200

CBC reaches the MIP's own true optimum; CP-SAT beats that bound by 5.56% while scheduling one more case – it's searching a strictly larger, correct feasible region, not searching harder. Its own looser gap (6.62%) reflects that larger space, not a weaker search.

How it's built

- **CP-SAT** (`cp_sat_interval_solver.py`) – the model, default and only backend needed to see it run.
- **Comparison MIP** (`milp_baseline_solver.py`) – CBC / SCIP / Gurobi / CPLEX, kept only as the empirical check on the CP-over-MIP argument.
- **CP Optimizer** (`cp_optimizer_solver.py`, optional, license-gated) – a second CP engine that can model sequence-dependent turnover; falls back to CP-SAT automatically.
- `tests/test_model.py` – the acceptance contract every solver's output is checked against (hard constraints, cross-solver agreement on the demo instance).

A reusable library of models

Four layers, solver-agnostic except the bottom one:

- 1 **Core data types** – plain dataclasses, no solver imports.
- 2 **Constraint patterns** – capacity sums, NoOverlap-based non-overlap, tiered-priority tardiness, eligibility pre-filters – reusable across nurse rostering, bed allocation, this project.
- 3 **Problem templates** – compose those patterns into a specific formulation. This project is one template.
- 4 **Solver-adapter layer** – one file per backend family (MILP, CP, local search), so a new problem picks a backend without rewriting how constraints are expressed.

The CP-vs-MIP comparison this project runs end to end is itself the template for layer 4: argue the backend choice from the problem's structure, then check it empirically, before committing.

Passing the torch

- FORMULATION.md + FORMULATION_CP.md – the math, one source of truth.
- src/model/types.py – the dataclasses *are* the data dictionary; every symbol maps to a field.
- The solver code – every constraint carries the same C-number as the math, read side by side.
- tests/test_model.py – the acceptance bar; a new test alongside any new constraint, not after it.
- A short glossary – most confusion on a project like this is vocabulary (“room roster”, “ambulatory”), not the math.

Where I'd take this next

Extension	Approach
Stochastic durations	Two-stage stochastic program
Same-day rescheduling	LNS seeded from the current plan
Explicit emergency reserve	Carve out a per-day capacity block
Nurse/anaesthetist rostering	Extend H , reuse the surgeon pattern
Multi-week rolling horizon	Solve weekly, carry forward at bumped priority
Day-varying bed capacity	Per-day-segmented cumulative resource
Per-specialty fairness	Secondary objective bounding overdue share

Questions

Thank you.

▸ [Back to agenda](#)

▸ [Jump to full math \(backup\)](#)

Backup A – full parameters, and where each default comes from

- **Turnover buckets** (15/25/40 min by duration) – a heuristic proxy for the real 15–60 min range; replace with measured hospital data.
- **Surgeon limits** (240/960 in demo) – one half-day theatre session (Cardoen et al., 2010); weekly figure leaves room for clinics/ward rounds.
- **Max waits** (270/60/15/3 days) – Portugal's SIGIC policy, used as an evidence-based starting point.
- **Priority multipliers** (1/4.5/18/90) – *derived*, not invented: see Backup B.
- **Equipment/bed capacity** – sized tight enough to make the demo exercise the constraint, not measured from any one hospital.

Backup B – the priority multiplier is a derived ratio

$$\mu_p = \frac{wl_1^{max}}{wl_p^{max}} \Rightarrow \mu_2 = \frac{270}{60} = 4.5, \quad \mu_3 = \frac{270}{15} = 18, \quad \mu_4 = \frac{270}{3} = 90$$

A tier allowed only a twelfth of priority-1's wait (15 vs. 270 days) is, by the policy's own design, roughly twelve times more wait-sensitive – weighting its breach days twelve times as heavily is consistent with the same policy that set the wait targets, not a second independent judgment call.

Caveat: μ_p is keyed to priority *tier*, not overdue severity. Raising it uniformly protects high-tier cases generally, not specifically the most-overdue ones – a per-service fairness target is the right fix if that shows up in practice, not a bigger global multiplier.

[◀ Agenda](#)

Backup C – the displacement margin, derived

Term 3 (w_c) must dominate every Term-1/2 coefficient, or the model could prefer dropping a schedulable case to dodge a tardiness charge.

$$\text{margin}_{\min} = 1 + \frac{\alpha \cdot n_{\text{days}}}{\max_c dd_c}$$

With $\alpha = 2$, $n_{\text{days}} = 5$, $\max_c dd_c \approx 270$:

$$\text{margin}_{\min} \approx 1.037$$

The model uses **1.2** – clears the minimum with real margin to spare, since $\max_c dd_c$ shrinks on instances with only short-horizon, high-priority cases, and a fixed margin has to stay safe across that range.

α itself appears only in Term 2's day coefficient – it can only change *which day* an overdue case lands on, never *how many* cases get scheduled.

Backup D – CP-SAT decision variables, in full

For every (c, d, r) surviving eligibility:

$$\text{pr}_{cdr} \in \{0, 1\}, \quad \text{start}_{cdr}, \text{end}_{cdr} \in [0, k_{dr}]$$

$$\text{iv}_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{\text{tot}}, \text{end}_{cdr}, \text{pr}_{cdr})$$

$$\text{sgiv}_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{\text{op}}, \text{sgend}_{cdr}, \text{pr}_{cdr})$$

Both share start ; both are *present* (actually constrain their resource) exactly when $\text{pr}_{cdr} = 1$. For every non-priority-4 case:

$$u_c \in \{0, 1\}, \quad \sum_{d,r} \text{pr}_{cdr} + u_c = 1$$

Backup E – the objective, in full

$$w_c = \mu_{p_c} \cdot \text{PenaltyCurve}(dd_c) + 1.2 \cdot \max_{c' \in C} dd_{c'}$$

`PenaltyCurve` (`src/model/penalty.py`) is flat while a case has slack, escalates sharply once it crosses its deadline, and keeps climbing the longer it stays overdue. μ_{p_c} scales the curve's *output* once – never the curve's input, and never reapplied a second time anywhere else in the codebase.

CP-SAT requires integer coefficients; each term is rounded at build time – negligible at the coefficient magnitudes here (tens to low thousands).

Backup F – constraints C1–C11, in full (1/2)

$$\mathbf{C1:} \quad \sum_{c:\text{patient}(c)=n} \sum_{d,r} \text{pr}_{cdr} \leq 1 \quad \forall n$$

C2: $\sum_r \text{pr}_{c,1,r} = 1$ for priority-4 cases; all other slots forced to 0.

C7 (room, exact):

$$\text{AddNoOverlap}(\{\text{iv}_{cdr} : d, r \text{ fixed}\}) \quad \forall d, r$$

C8 (surgeon, exact + daily cap):

$$\text{AddNoOverlap}(\{\text{sgiv}_{cdr}\}), \quad \sum_{c:\text{surgeon}(c)=h} \sum_r t_c^{op} \text{pr}_{cdr} \leq k_{hd}$$

$$\mathbf{C9:} \quad \sum_{c:\text{surgeon}(c)=h} \sum_{d,r} t_c^{op} \text{pr}_{cdr} \leq k_h \quad \forall h$$

Backup F – constraints C1–C11, in full (2/2)

C10 (equipment, exact concurrency):

$$\text{AddCumulative}(\{\text{iv}_{cdr} : u_{ce} = 1, d \text{ fixed}\}, 1, \kappa_{ed}) \quad \forall e, d$$

C11 (beds, with overflow):

$$\text{bed}_c = \text{NewOptionalIntervalVar}(\text{dayof}_c, \text{los}_c, \text{dayof}_c + \text{los}_c, 1 - u_c)$$

$$\text{AddCumulative}(\{\text{bed}_c : \rho(c) = \rho\}, 1, \beta_\rho) \quad \forall \rho$$

$$\text{overflow}_c = \max(0, (\text{dayof}_c + \text{los}_c) - n_{\text{days}})$$

A multi-day stay needs a real “day of surgery” value to start counting from – the concrete reason this model is interval-based at all.

◀ Agenda

Backup G – the comparison MIP, in full

$$\begin{aligned}
 & x_{cdr} \in \{0, 1\} \quad \forall c, d \in D_c, r \quad z_c \in [0, 1] \quad \forall c : p_c \neq 4 \\
 \min \quad & \sum_{c: dd_c \geq 0} \sum_{d, r} [dd_c + d] x_{cdr} + \sum_{c: dd_c < 0} \sum_{d, r} [dd_c + \alpha d] x_{cdr} + \sum_{c: p_c \neq 4} w_c z_c
 \end{aligned}$$

Same C1–C6, C9 as CP-SAT. Diverges at:

$$\begin{aligned}
 \text{C7 (sum):} \quad & \sum_c t_c^{\text{tot}} x_{cdr} \leq k_{dr} \quad \text{C10 (headcount):} \quad \sum_{c: u_{ce}=1} \sum_r x_{cdr} \leq \kappa_{ed}
 \end{aligned}$$

No C11 – a day-bucket model has no value that means “day of surgery” for a multi-day bed stay to start from. This, more than the equipment gap, is why this project ended up CP-based.

Backup H – CP Optimizer (optional second CP engine)

Kept for one reason: sequence-dependent turnover, which CP-SAT can't express without restructuring its intervals.

	Same service, back-to-back	Different service, back-to-back
CP-SAT (duration-bucketed)	charged on the case alone	same
CP Optimizer (transition matrix)	15 min	35 min

Demo instance: identical objective to CP-SAT (155.0, 20/20). At the 200-case scale its own gap closes far more slowly – a search-tuning gap (no warm start applied), not a modeling one. Stays an appendix backend, not the model.

[◀ Agenda](#)

Backup I – references

- 1 Cardoen, Demeulemeester & Beliën (2010). EJOR 201(3), 921–932.
- 2 Marques & Captivo (2015). MSc thesis, Universidade de Lisboa.
- 3 Denton, Miller, Balasubramanian & Huschka (2010). Oper. Res. 58(4), 802–816.
- 4 SIGIC – Portaria n.º 45/2008, Portugal.
- 5 Van Riet & Demeulemeester (2015). OR Spectrum 37(1), 59–87.
- 6 Macario (2010). J. Clin. Anesth. 22(4), 233–236.
- 7 Akbarzadeh & Maenhout (2023). Mendeley Data, V2/V1.
- 8 Perron & Furnon. CP-SAT: a Constraint Programming Solver, OR-Tools docs.
- 9 Baptiste, Le Pape & Nuijten (2001). Constraint-Based Scheduling, Kluwer.
- 10 Vilím (2004). CPAIOR 2004.
- 11 Schutt, Feydy, Stuckey & Wallace (2009). CP 2009.
- 12 Laborie (2009). CPAIOR 2009.